Capitolo 1

Il Modello ad Attori e Akka

1.1 Fondamenti della Computazione Concorrente e Distribuita

1.1.1 La Crisi della Concorrenza e il Paradigma a Memoria Condivisa

Le architetture computazionali moderne sono intrinsecamente parallele. Il raggiungimento dei limiti fisici di dissipazione termica e di densità di potenza (il cosiddetto *"power wall"*) ha imposto l'arresto della scalabilità della frequenza di clock dei singoli processori. L'industria dell'hardware ha risposto deviando lo sviluppo verso architetture multi-core, many-core e sistemi distribuiti su larga scala. Di conseguenza, l'ottenimento di un elevato throughput e di una bassa latenza nell'elaborazione del software richiede l'adozione di modelli di esecuzione concorrente.

Storicamente, l'astrazione dominante per la programmazione concorrente su sistemi a memoria condivisa (*shared-memory*) è stata il multi-threading affiancato da primitive esplicite di sincronizzazione (quali lock di mutua esclusione, semafori, monitor e operazioni atomiche in memoria). In tale modello, più thread concorrenti condividono un unico spazio di indirizzamento virtuale, accedendo e mutando direttamente le medesime locazioni di memoria. Tuttavia, questo approccio manifesta gravi patologie sia teoriche sia ingegneristiche:

1. **Interleaving Non Deterministico e Race Condition**: L'ordine temporale degli accessi concorrenti alla memoria è governato dallo scheduler dei thread del sistema operativo e dai modelli di memoria hardware (store buffer, esecuzione fuori ordine e ottimizzazioni del compilatore). Se due o più thread accedono contemporaneamente alla medesima locazione di memoria senza opportuna sincronizzazione e almeno un accesso è in scrittura, si verifica una **race condition** (corsa critica). Rilevare, riprodurre e risolvere tali anomalie è notoriamente complesso a causa della natura non riproducibile dei diversi interleaving di esecuzione.

2. **Deadlock, Livelock e Contesa dei Lock**: Il coordinamento degli accessi a risorse condivise multiple impone gerarchie globali e rigorose di acquisizione dei lock per scongiurare condizioni di attesa circolare (le note condizioni di Coffman per il deadlock). All'aumentare della complessità del sistema, mantenere coerenti tali gerarchie diviene intrattabile. L'adozione di lock a grana fine (*fine-grained locking*) introduce elevata latenza di sincronizzazione, contesa di cache (cache bouncing) e alto rischio di deadlock, mentre lock a grana grossa (*coarse-grained locking*) serializzano l'esecuzione, distruggendo i vantaggi prestazionali del parallelismo.

3. **Violazione dell'Incapsulamento Object-Oriented**: Nella programmazione a oggetti (OOP), un oggetto incapsula stato e invarianti esponendo metodi pubblici per manipolarlo. In ambiente concorrente a memoria condivisa, i confini dell'oggetto risultano del tutto porosi: più thread possono invocare simultaneamente i metodi dell'oggetto, mutandone i campi interni senza alcun isolamento intrinseco. L'incapsulamento OOP fallisce pertanto nel proteggere le invarianti di stato in presenza di accessi concorrenti.

4. **Assenza di Portabilità Spaziale e di Rete**: Le primitive a memoria condivisa dipendono strettamente dalla RAM fisica condivisa, da istruzioni atomiche hardware (es. Compare-And-Swap - CAS) e da protocolli di coerenza delle cache. Tali primitive non possono varcare i confini fisici di una scheda madre o di una rete locale. Di conseguenza, il codice strutturato su lock non è trasferibile su cluster distribuiti senza una riscrittura totale dell'architettura applicativa.

1.1.2 Il Modello ad Attori: Paradigma Matematico di Concorrenza Guidata da Messaggi

Al fine di superare i limiti strutturali del multi-threading a memoria condivisa, Carl Hewitt, Peter Bishop e Richard Steiger formularono nel 1973 il **Modello ad Attori** (*Actor Model*), successivamente formalizzato ed esteso da Gul Agha nel 1986. Il modello assume l'**Attore** quale primitiva universale della computazione concorrente e distribuita.

Joe Armstrong, co-inventore del linguaggio Erlang, sintetizzò il principio fondante di questo paradigma:

> *«Il mondo è parallelo. Se vogliamo scrivere programmi che si comportino come gli oggetti nel mondo reale, tali programmi dovranno avere una struttura concorrente. Usando un linguaggio progettato per applicazioni concorrenti, lo sviluppo diventa straordinariamente più semplice. I programmi Erlang modellano il modo in cui pensiamo e interagiamo.»*

A completamento, la massima ingegneristica dell'*Effective Go*, pienamente adottata nei sistemi ad attori, sancisce:

> *«Non comunicare condividendo la memoria; al contrario, condividi la memoria comunicando.»*

Nel modello ad attori formale, un attore è un'entità computazionale autonoma che incapsula:

- **Stato Privato (*Private State*)**: Le variabili interne dell'attore sono strettamente confinate al suo contesto di esecuzione. Nessun'altra entità nel sistema può leggere o modificare direttamente lo stato interno di un attore.

- **Una Coda di Messaggi (*Mailbox*)**: Una struttura a coda associata all'attore in cui i messaggi in ingresso vengono memorizzati in attesa di elaborazione.

- **Un Comportamento (*Behavior / Message Handler*)**: Una funzione che definisce la reazione logica dell'attore all'arrivo di ciascun messaggio.

Formalmente, in risposta a un messaggio prelevato dalla propria mailbox, un attore può compiere esclusivamente una combinazione finita delle seguenti operazioni:

$$\mathcal{A}(\text{Messaggio}) \longrightarrow \begin{cases} 1. & \text{Creare un insieme finito di nuovi attori: } \{\mathcal{A}'_1, \mathcal{A}'_2, \ldots, \mathcal{A}'_k\}, \\ 2. & \text{Inviare un insieme finito di messaggi ad attori noti: } \{(\text{Target}_i, \text{Msg}_i)\}_{i=1}^{m}, \\ 3. & \text{Designare il comportamento/stato successivo } \beta_{\text{next}} \text{ per il prossimo messaggio.} \end{cases} \tag{1.1}$$

Actors basics

```java
public class CounterActor extends AbstractActor {
  private int counter;

  public CounterActor() {
    this.counter = 0;
  }

  @Override
  public Receive createReceive() {
    return receiveBuilder() //
        .match(SimpleMessage.class, this::onMessage) //
        .build();
  }

  void onMessage(SimpleMessage message) {
    ++counter;
    System.out.println("Counter increased to: " + counter);
  }

  static Props props() {
    return Props.create(CounterActor.class);
  }

}
```

Figura 1.1: Struttura architetturale di un Attore: i messaggi in arrivo si accodano nella Mailbox privata e vengono elaborati in sequenza senza interferenze concorrenti.

1.1.3 Invarianti Fondamentali e Proprietà del Modello

Le regole operative del modello ad attori garantiscono proprietà essenziali:

- **Confinamento di Thread e Assenza di Stato Mutabile Condiviso**: Gli attori non condividono mai variabili. La comunicazione avviene esclusivamente tramite scambio asincrono di messaggi. Poiché ciascun attore elabora un singolo messaggio alla volta in modo sequenziale dalla propria mailbox, il corpo dell'attore è totalmente privo di corse critiche interne. Non occorre alcun blocco sincronizzato, mutex o primitiva atomica all'interno della logica di business.

- **Elaborazione Atomica del Messaggio**: Il ciclo di vita dell'elaborazione di un singolo messaggio è atomico rispetto allo stato dell'attore: il messaggio viene elaborato dall'inizio alla fine senza interleaving con altri messaggi indirizzati al medesimo attore. Eventuali messaggi ricevuti nel frattempo rimangono in coda nella mailbox.

- **Asincronismo Nativamente Non Bloccante**: La trasmissione dei messaggi è nativamente asincrona. Quando l'Attore A invia un messaggio all'Attore B, A non si blocca in attesa che B lo riceva o lo elabori, ma prosegue immediatamente la propria esecuzione.
- **Stato e Persistenza**: Gli attori mantengono lo stato tra invocazioni successive. In estensioni persistenti (es. Akka Persistence), lo stato può essere salvato su supporti durevoli mediante tecniche di *event sourcing* o snapshotting.
- **Trasparenza Spaziale e Topologica**: Gli attori interagiscono tramite riferimenti logici astratti (**ActorRef**), slegati dagli indirizzi di memoria fisici o dai descrittori di socket. La sintassi di comunicazione è identica sia che l'attore risieda sullo stesso core, nello stesso processo JVM, su una macchina differente della LAN o in un data center remoto.

1.1.4 Adozione Industriale

Il modello ad attori non costituisce una mera formulazione teorica, ma la spina dorsale di sistemi industriali ad altissima disponibilità e scalabilità:

- **Erlang / OTP**: Sviluppato in Ericsson da Joe Armstrong per gestire centrali telefoniche con disponibilità richiesta del 99.9999999% ("nove nove"). Costituisce l'infrastruttura di WhatsApp (miliardi di connessioni concorrenti con costi hardware minimi), Apache CouchDB e RabbitMQ.
- **Microsoft Orleans**: Framework a "virtual actors" alla base dei servizi cloud distribuiti per il multiplayer di videogiochi su scala globale come *Halo*.
- **Akka (JVM - Java e Scala)**: Progettato da Jonas Bonér e Lightbend, fornisce un runtime per la Java Virtual Machine impiegato in sistemi di produzione mission-critical da aziende leader come BMW, Volkswagen Group, Siemens, Bosch e Volvo.

—

1.2 L'Architettura di Akka su Piattaforma JVM

1.2.1 Componenti Fondamentali di Akka

In ambiente JVM, Akka concretizza il modello ad attori mediante quattro astrazioni chiave:

1. **ActorSystem**: Contenitore pesante e contesto radice dell'applicazione ad attori. Alloca e governa le risorse condivise, tra cui thread pool, dispatcher, parametri di configurazione, la mailbox dei Dead Letters e i servizi di scheduling. Tipicamente si istanzia un solo `ActorSystem` per processo JVM:

```
ActorSystem system = ActorSystem.create("NSDS-System");

```

2. **Props**: Oggetto immutabile di configurazione che incapsula la formula per la creazione di un attore, specificando la classe di appartenenza, i parametri del costruttore e il dispatcher designato.

3. **ActorRef**: Riferimento logico immutabile e serializzabile all'attore. Il codice client e gli attori pari non interagiscono mai direttamente con l'oggetto Java concreto; ogni interazione è mediata dall'`ActorRef`. Un `ActorRef` rimane valido anche in caso di restart dell'attore a seguito di fallimento.

4. **AbstractActor**: La classe base Java da estendere per definire la logica di ricezione dei messaggi mediante il metodo `createReceive()`.

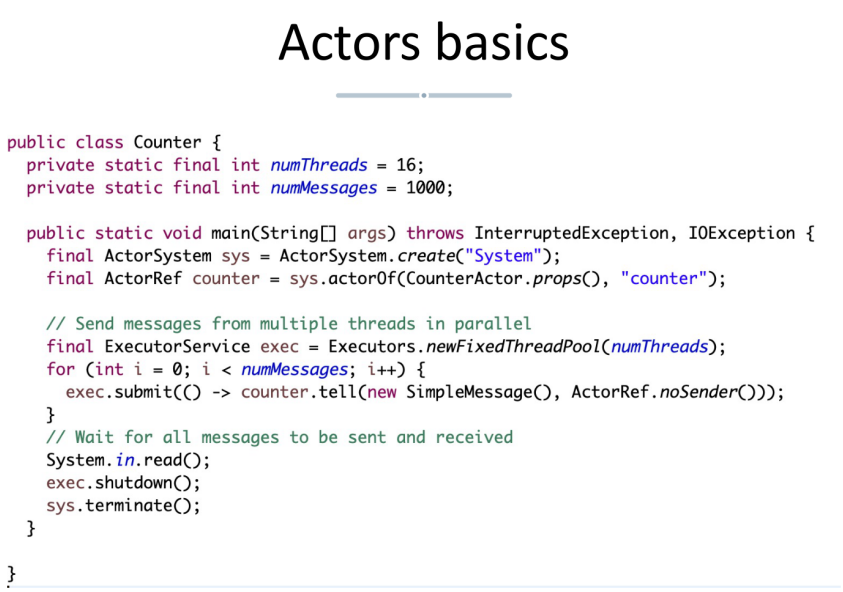


Figura 1.2: Disaccoppiamento tra ActorRef, Mailbox e istanza dell'attore: l'interazione è sempre mediata dall'ActorRef a tutela dell'isolamento dello stato.

1.2.2 Istanziazione degli Attori e Pattern Statico Props

Per creare un attore, si invoca il metodo factory `actorOf` sull'`ActorSystem` (per attori top-level) o sul contesto locale `getContext()` (per attori figli), specificando un'istanza di `Props` e un identificatore univoco testuale.

È raccomandato definire un metodo factory statico `props()` all'interno della classe dell'attore:

```
1  public class CounterActor extends AbstractActor {
2      private int counter;
3
4      public CounterActor(int initialValue) {
5          this.counter = initialValue;
6      }
7
8      // Factory method statico per Props
9      public static Props props(int initialValue) {
10         return Props.create(CounterActor.class, () -> new CounterActor(
       initialValue));
11     }
12
13     @Override
14     public Receive createReceive() {
15         return receiveBuilder()
16             .match(IncrementMessage.class, msg -> {
```

```
                this.counter += msg.getDelta();
            })
            .build();
    }
}
```

Nel codice principale:

```
ActorSystem system = ActorSystem.create("ClusterSystem");
ActorRef counterRef = system.actorOf(CounterActor.props(0), "counterActor")
    ;
```

1.2.3 Ciclo di Vita dell'Attore e Hook di Callback

Un attore in Akka attraversa diverse fasi del ciclo di vita: **Starting**, **Receiving** / **Executing**, **Restarting** (in caso di guasto) e **Stopped**. Il framework mette a disposizione callback espliciti per la gestione delle risorse:

Metodo di Lifecycle	**Momento di Invocazione**	**Semantica Operativa e Utilizzo Tipico**
`preStart()`	Subito dopo la creazione dell'attore, prima di elaborare il primo messaggio.	Inizializzazione dello stato privato, apertura di socket o connessioni a database, pianificazione di messaggi periodici.
`preRestart(reason, message)`	Invocato sull'istanza che ha fallito, prima della sua dismissione per riavvio.	Rilascio di risorse esterne; per default termina ricorsivamente tutti gli attori figli e invoca `postStop()`.
`postRestart(reason)`	Invocato sulla nuova istanza subito dopo la ri-creazione a seguito di riavvio.	Re-inizializzazione dello stato per la nuova istanza; per default invoca `preStart()`.
`postStop()`	Invocato dopo che l'attore è stato arrestato definitivamente.	Pulizia finale, rilascio di handle di sistema, notifica ai monitor remoti, salvataggio di buffer residui su disco.

Tabella 1.1: Callback del Ciclo di Vita di un Attore Akka.

1.2.4 Semantica della Mailbox e Requisito di Immutabilità

Ogni attore possiede una propria **Mailbox**. L'implementazione di default è a capacità illimitata (*unbounded*) e adotta una disciplina di accodamento **FIFO** per ciascuna coppia mittente-destinatario.

Garanzie di Ordinamento

In Akka, l'ordinamento soddisfa due garanzie formali:

1. **Ordinamento FIFO tra Coppia Mittente-Destinatario**: Se l'Attore A invia il messaggio M_1 e successivamente M_2 all'Attore B, M_1 sarà sempre recapitato ed

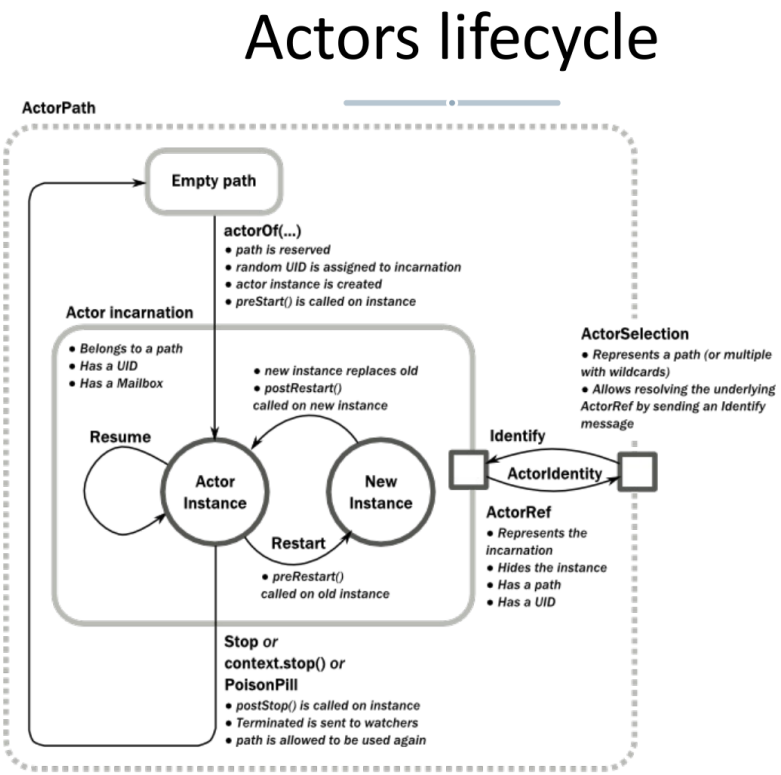


Figura 1.3: Transizioni di stato del ciclo di vita di un attore in Akka, con evidenza del ciclo di riavvio controllato dal supervisore.

elaborato da B prima di M_2, a condizione che la trasmissione avvenga localmente o su connessione di rete affidabile punto-a-punto (come TCP):

$$\text{Invio}_A(M_1 \prec M_2) \implies \text{Ricezione}_B(M_1 \prec M_2) \tag{1.2}$$

2. **Interleaving Non Deterministico da Mittenti Diversi**: Se l'Attore A trasmette M_A all'Attore C e l'Attore B trasmette M_B all'Attore C, non esiste alcun vincolo di ordine relativo prefissato tra M_A e M_B. L'ordine di accodamento dipenderà esclusivamente da latenze di rete e scheduling dei thread.

Il Vincolo di Immutabilità dei Messaggi

Poiché gli attori residenti sulla medesima JVM condividono lo heap, i riferimenti agli oggetti messaggio vengono trasmessi come puntatori diretti per azzerare l'overhead di serializzazione. Ne deriva una regola categorica:

> **Tutti i messaggi scambiati tra attori DEVONO essere rigorosamente immutabili.**

Se un messaggio contenesse campi mutabili (es. setter pubblici, array o collezioni modificabili), thread differenti in esecuzione concorrente potrebbero leggere e mutare simultaneamente il carico utile, reintroducendo le medesime race condition e anomalie di memoria che il modello ad attori si prefigge di eliminare.

In Java, l'immutabilità va garantita a livello di codice con la seguente struttura:

```java
// Pattern canonico per un Messaggio Immutabile
public final class TemperatureReadingMsg implements Serializable {
    private final String sensorId;
    private final double temperature;
    private final long timestamp;

    public TemperatureReadingMsg(String sensorId, double temperature, long
    timestamp) {
        this.sensorId = sensorId;
        this.temperature = temperature;
        this.timestamp = timestamp;
    }

    public String getSensorId() { return sensorId; }
    public double getTemperature() { return temperature; }
    public long getTimestamp() { return timestamp; }
}
```

Messaggi Non Gestiti e Dead Letters

All'interno del blocco `createReceive()`, le clausole di matching vengono valutate nell'ordine sequenziale di dichiarazione. Qualora un messaggio in arrivo non soddisfi alcuna clausola definita, Akka invoca il gestore `unhandled(Object message)`, che per default inoltra il messaggio allo stream di eventi dei **Dead Letters** (*Lettere Morte*). Anche i messaggi inviati ad attori arrestati confluiscono nei Dead Letters.

—

1.3 Pattern di Comunicazione, Protocolli e Trasparenza di Posizione

1.3.1 Il Pattern Tell (Asincrono Fire-and-Forget)

La primitiva fondamentale per la trasmissione di messaggi è `tell()`:

```java
targetActorRef.tell(message, senderActorRef);
```

L'operazione è asincrona e non bloccante: deposita il messaggio nella mailbox del destinatario e restituisce immediatamente il controllo al thread chiamante.

Regole operative:

- **Identificazione del Mittente**: Se l'invio avviene dall'interno di un attore, specificare `self()` come secondo parametro consente al destinatario di replicare tramite `sender().tell(replyMsg, self())`.
- **Assenza di Mittente**: Se l'invio avviene dall'esterno del contesto attori (es. dal metodo `main`), si utilizza `ActorRef.noSender()`:

```java
counterActor.tell(new IncrementMessage(1), ActorRef.noSender());

```

- **Inoltro con Forwarding**: Un attore può inoltrare un messaggio a un terzo attore preservando l'indirizzo originale del mittente originario mediante `forward()`:

```java
workerActor.forward(message, getContext());

```

1.3.2 Il Pattern Ask (Richiesta-Risposta con Future)

Sebbene il `tell()` rappresenti il pattern puro, vi sono scenari in cui occorre interagire con confini applicativi non basati su attori (es. servire richieste HTTP sincrone, orchestrare test di integrazione o interrogare un attore dal `main`). In tali contesti si impiega il **Pattern Ask** via `akka.pattern.Patterns.ask`:

```
import static akka.pattern.Patterns.ask;
import scala.concurrent.Future;
import scala.concurrent.Await;
import scala.concurrent.duration.Duration;
import java.util.concurrent.TimeUnit;

Duration timeout = Duration.create(3, TimeUnit.SECONDS);
Future<Object> future = ask(targetActorRef, new QueryBalanceMsg(accountId),
    3000);

// Attesa bloccante (da limitare a test o confini esterni)
BalanceResponse reply = (BalanceResponse) Await.result(future, timeout);
```

Meccanismo Interno e Costi Computazionali di Ask

Internamente, il pattern Ask opera come segue:

1. Akka alloca un **Promise Actor** temporaneo e usa-e-getta all'interno dell'`ActorSystem`.
2. Il messaggio viene trasmesso al destinatario indicando il Promise Actor come `sender()`.
3. Viene registrato un timer presso il servizio di scheduling per gestire il timeout prefissato.
4. Quando il destinatario invoca `sender().tell(response, self())`, il Promise Actor intercetta la risposta, completa la `Future` e viene terminato.
5. Se il timeout scade prima della risposta, la Future fallisce con `AskTimeoutException`.

Dimensione	**Pattern Tell (`tell`)**	**Pattern Ask (`ask`)**
Modello di Esecuzione	Totalmente asincrono, non bloccante (*fire-and-forget*).	Asincrono basato su Future; bloccabile tramite `Await.result()`.
Overhead di Risorse	Minimo: inserimento diretto nella mailbox del destinatario.	Medio: istanzia un Promise Actor temporaneo, un timer e un descrittore Future.
Gestione Guasti	Trasparente: l'eventuale perdita o crash è gestita dal supervisore o via Dead Letters.	Esplicita: fallisce con `AskTimeoutException` in caso di timeout o mancata replica.
Ambito Elettivo	Comunicazione ordinaria interna tra attori, pipeline ad alto throughput.	Interfacciamento con contesti esterni non ad attori (REST API, metodi di test `main`).

Tabella 1.2: Analisi comparativa: Pattern Tell rispetto a Pattern Ask.

1.3.3 Garanzie di Consegna: Best-Effort (At-Most-Once)

La teoria dei sistemi distribuiti dimostra che garantire la consegna esattamente una volta (*exactly-once*) su reti inaffidabili senza coordinamento globale è impossibile (il paradosso dei due generali).

Akka adotta per impostazione predefinita una semantica **at-most-once (best-effort)**:

- I messaggi possono andare perduti in caso di crash dell'attore, overflow della mailbox o partizione di rete.
- Il livello di trasporto non introduce duplicati arbitrari.

Motivazione Architetturale ("Less is More"): Come spiegato dal Prof. Luca Mottola, imporre garanzie rigide di consegna a livello di trasporto richiede un costo sproporzionato: sequenziamento dei pacchetti, acknowledgment, buffer di ritrasmissione, persistenza su disco e tabelle di deduplicazione. In ambiente distribuito:

1. Il protocollo IP è nativamente best-effort.
2. TCP fornisce flussi affidabili su singola connessione, ma non può mascherare crash di sistema, partizioni o riavvii dei processi.
3. Molti domini operativi gestiscono dati temporalmente deperibili o idempotenti (es. letture di sensori, dove un dato perso è reso obsoleto dal valore successivo).
4. I requisiti che esigono garanzie superiori possono essere soddisfatti implementando acknowledgment a livello applicativo o integrando code persistenti dedicate (come Apache Kafka).

1.3.4 Trasparenza di Posizione e Akka Remoting

La **Trasparenza di Posizione** (*Location Transparency*) costituisce uno dei massimi punti di forza di Akka: un `ActorRef` astrae la locazione fisica dell'attore. Il programmatore invia messaggi con la medesima sintassi `ref.tell(msg, self())` a prescindere dal fatto che il destinatario si trovi sulla JVM locale o su un server remoto.

Actor Path e Indirizzamento Gerarchico

Ogni attore è identificato univocamente da un URI gerarchico strutturato come segue:

```
akka.<protocollo>://<nome-actor-system>@<hostname>:<porta>/user/<percorso-gerarchico>
```
(1.3)

Esempi:

- Percorso locale: `akka://NSDS-System/user/supervisor/worker-1`
- Percorso remoto su TCP: `akka.tcp://ClusterNode@192.168.1.100:2552/user/dataProcessor`

Configurazione Remota via HOCON

I parametri di networking si configurano tramite file HOCON (`application.conf`):

```
akka {
  actor {
    provider = remote
  }
  remote {
    enabled-transports = ["akka.remote.netty.tcp"]
    netty.tcp {
      hostname = "127.0.0.1"
      port = 6123
    }
  }
}
```

Risoluzione di Attori Remoti: ActorSelection

Per contattare un attore remoto di cui non si possiede preventivamente l'`ActorRef`, si interroga l'URI logico tramite `actorSelection()`:

```
// Ricerca del server remoto da parte del client
String remoteAddress = "akka.tcp://ServerSystem@127.0.0.1:6123/user/
    serverActor";
ActorSelection remoteSelection = getContext().actorSelection(remoteAddress)
    ;

// Invio diretto alla selezione logica
remoteSelection.tell(new QueryMessage("data"), getSelf());
```

Al momento della risposta, il mittente remoto recapita il proprio `ActorRef` concreto all'interno del campo `sender()`, abilitando le successive comunicazioni dirette ad alte prestazioni.

—

1.4 Modifica Dinamica del Comportamento e Meccanismo di Stash

1.4.1 Macchine a Stati Finiti con Become e Unbecome

In molti protocolli distribuiti, un attore deve modificare radicalmente la propria logica di gestione dei messaggi in funzione dello stato operativo in cui si trova (es. transizione tra gli stati *Disconnesso*, *In Connessione*, *Attivo*, *Occupato* o *Sospeso*).

Anziché ricorrere a catene condizionali complesse e fragili (`if-else` o `switch` su flag di stato interni), Akka consente all'attore di sostituire l'intero blocco di ricezione a runtime mediante `getContext().become()`:

```
public class ProtocolActor extends AbstractActor {

    @Override
    public Receive createReceive() {
        return disconnectedBehavior();
    }

    private Receive disconnectedBehavior() {
        return receiveBuilder()
            .match(ConnectMsg.class, msg -> {
                System.out.println("Connessione a " + msg.getHost());
```

```
                getContext().become(connectedBehavior());
            })
            .match(DataMsg.class, msg -> {
                System.err.println("Dati rifiutati: attore non connesso!");
            })
            .build();
    }

    private Receive connectedBehavior() {
        return receiveBuilder()
            .match(DataMsg.class, msg -> {
                System.out.println("Dati elaborati: " + msg.getPayload());
            })
            .match(DisconnectMsg.class, msg -> {
                System.out.println("Disconnessione in corso...");
                getContext().become(disconnectedBehavior());
            })
            .build();
    }
}
```

Se si passa il valore booleano `false` come secondo argomento a `become(newReceive, false)`, Akka non scarta il vecchio comportamento, ma lo inserisce in uno stack interno. Invocando successivamente `getContext().unbecome()`, l'attore ripristina istantaneamente il comportamento precedente.

1.4.2 Il Buffer di Stash: AbstractActorWithStash

Capita frequentemente che un attore, in un dato stato, riceva messaggi che non può gestire immediatamente, ma che non devono essere persi. Il pattern di **Stash** consente all'attore di accantonare temporaneamente tali messaggi in un buffer secondario per poi rielaborarli al momento opportuno.

Per implementare lo stash, l'attore deve:

1. Estendere la classe `AbstractActorWithStash`.
2. Invocare il metodo `stash()` sui messaggi da differire: il messaggio corrente viene rimosso dal ciclo di elaborazione e appeso in coda allo stash.
3. Quando l'attore commuta in uno stato abilitato a processarli, invocare il metodo `unstashAll()`.

Riposizionamento dei Messaggi nella Mailbox

All'invocazione di `unstashAll()`, Akka preleva tutti i messaggi accumulati nello stash e li reinserisce nella mailbox:

> **I messaggi estratti dallo Stash vengono reinseriti in TESTA alla Mailbox primaria, preservando l'ordine relativo con cui sono stati accantonati.**

Questo meccanismo assicura che i messaggi differiti vengano elaborati con priorità rispetto a nuovi messaggi sopraggiunti nel frattempo.

—

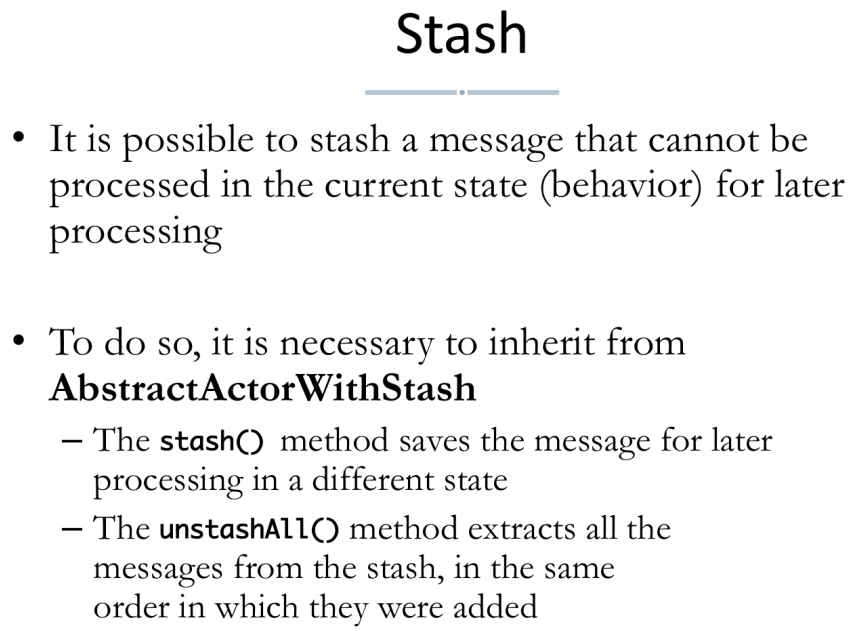


Figura 1.4: Funzionamento dello Stash: accantonamento temporaneo e reinserimento prioritario in testa alla Mailbox principale al momento del cambio di stato.

1.5 Tolleranza ai Guasti e Gerarchia di Supervisione

1.5.1 La Filosofia "Let It Crash"

Nei linguaggi orientati agli oggetti tradizionali (come Java standard), la gestione delle eccezioni è intrecciata alla logica funzionale mediante blocchi difensivi `try-catch`. Se un'eccezione non prevista sale lungo lo stack di chiamate, il thread muore in modo incontrollato, rischiando di lasciare strutture dati corrotte o lock bloccati per sempre.

Il modello ad attori sposa la filosofia di Joe Armstrong nota come **"Let It Crash"** (*lascia che si guasti*):

- **Separazione tra Logica di Business e Gestione Errori**: Lo sviluppatore non implementa codice difensivo per intercettare anomalie fatali all'interno dell'elaborazione funzionale. Se si verifica un'eccezione non recuperabile localmente, l'attore lancia l'errore e si arresta.
- **Supervisione Esterna**: La gestione del guasto è interamente esternalizzata. Il genitore gerarchico dell'attore—il suo **Supervisore**—viene notificato automaticamente dal runtime e applica una direttiva strategica di ripristino.

1.5.2 La Struttura Gerarchica degli Attori

In Akka, nessun attore vive isolato: essi sono organizzati in un albero gerarchico rigoroso. Ogni attore creato tramite `getContext().actorOf()` è figlio dell'attore che lo ha generato.

La gerarchia poggia su tre vertici principali:

1. **Root Guardian (`/`)**: Vertice supremo dell'intero runtime.
2. **System Guardian (`/system`)**: Sovrintende ai servizi infrastrutturali di framework (logging, socket di remoting, dead letters).

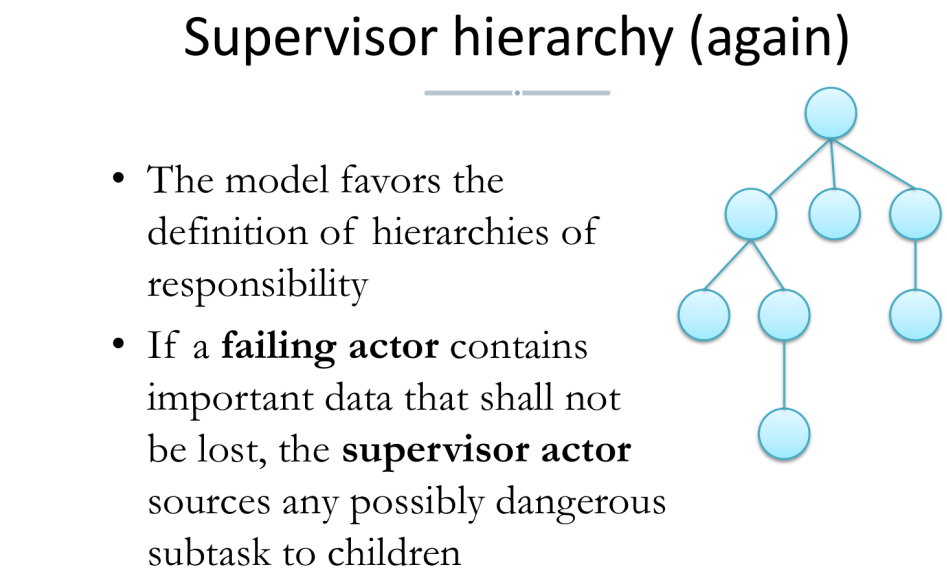


Figura 1.5: Albero di Supervisione in Akka: suddivisione tra guardiano di sistema, guardiano utente (/user) e gerarchia applicativa.

3. **User Guardian (`/user`)**: Genitore diretto di tutti gli attori applicativi creati via `system.actorOf()`.

1.5.3 Strategie di Supervisione e Direttive di Recupero

Un attore supervisore stabilisce la propria politica di resilienza sovrascrivendo il metodo `supervisorStrategy()`, restituendo un'istanza di `OneForOneStrategy` o di `AllForOneStrategy`:

- **OneForOneStrategy**: La direttiva di recupero scelta dal supervisore viene applicata **esclusivamente** all'attore figlio che ha sollevato l'errore. Gli altri attori figli continuano la loro normale attività senza interruzione. È la strategia d'elezione per carichi di lavoro indipendenti.
- **AllForOneStrategy**: La direttiva di recupero viene applicata a **tutti** gli attori figli del supervisore, anche se uno solo di essi ha manifestato il guasto. Questa strategia è indispensabile qualora i figli collaborino a una transazione coordinata in cui il guasto di un membro rende nullo o incoerente lo stato dei compagni.

Le Quattro Direttive di Supervisione

All'intercettazione del fallimento di un figlio, il supervisore applica una tra quattro direttive canoniche:

1. **Resume**: L'attore guasto **non** viene terminato. Il suo stato interno in memoria viene integralmente conservato e l'attore procede immediatamente all'elaborazione del messaggio successivo presente nella mailbox.
   - *Caso d'uso*: Errori transitori o non correlati a corruzione di stato (es. un pacchetto con formato non valido da scartare, un temporaneo timeout di rete).
2. **Restart** (*Politica di Default*): L'istanza dell'attore che ha fallito viene distrutta e una nuova istanza viene generata a partire dai `Props` originali.

- *Semantica di Stato*: Lo stato privato viene integralmente ripristinato ai valori iniziali del costruttore.
- *Semantica di Mailbox*: La mailbox viene **preservata**. I messaggi pendenti rimangono in coda e verranno elaborati dalla nuova istanza appena creata.
- *Hook Invocati*: Sulla vecchia istanza si esegue `preRestart(reason, message)`; sulla nuova istanza si esegue `postRestart(reason)`.
- *Caso d'uso*: Corruzione di stato interno o eccezioni inattese non gestibili localmente.

3. **Stop**: L'attore viene arrestato definitivamente. La mailbox viene svuotata (i messaggi residui finiscono nei Dead Letters), i suoi eventuali figli vengono ricorsivamente terminati e il relativo `ActorRef` cessa di essere utilizzabile.
   - *Caso d'uso*: Esaurimento fatale di risorse o errori irreversibili di configurazione.
4. **Escalate**: Il supervisore riconosce di non avere la competenza logica per gestire quel tipo di eccezione; sospende se stesso e tutti i suoi figli e ri-lancia l'errore al proprio genitore lungo la gerarchia.

Implementazione della Strategia in Java

Il codice seguente illustra la configurazione di una `OneForOneStrategy` con limiti sul numero massimo di tentativi di riavvio in una finestra temporale mobile:

```
import akka.actor.AbstractActor;
import akka.actor.OneForOneStrategy;
import akka.actor.SupervisorStrategy;
import akka.japi.pf.DeciderBuilder;
import java.time.Duration;

public class CustomSupervisorActor extends AbstractActor {

    // Massimo 10 riavvii consentiti nell'arco di 1 minuto
    private static final SupervisorStrategy STRATEGY = new
    OneForOneStrategy(
        10,
        Duration.ofMinutes(1),
        DeciderBuilder
            .match(IllegalArgumentException.class, e -> SupervisorStrategy.
    resume())
            .match(NullPointerException.class, e -> SupervisorStrategy.
    restart())
            .match(IllegalStateException.class, e -> SupervisorStrategy.
    stop())
            .matchAny(e -> SupervisorStrategy.escalate())
            .build()
    );

    @Override
    public SupervisorStrategy supervisorStrategy() {
        return STRATEGY;
    }

    @Override
    public Receive createReceive() {
        return receiveBuilder()
            .match(Props.class, childProps -> {
```

```
30                  ActorRef child = getContext().actorOf(childProps);
31                  sender().tell(child, self());
32              })
33              .build();
34      }
35  }
```

—

1.6 Akka Clustering e Topologie Distribuite

1.6.1 Architettura Decentralizzata di Cluster Membership

Akka Cluster estende la trasparenza di posizione su un insieme dinamico di macchine di rete senza fare affidamento su coordinatori centralizzati (quali Apache ZooKeeper):

- **Topologia Peer-to-Peer Simmetrica**: Tutti i nodi partecipano su base paritaria; non esiste alcun singolo punto di fallimento (*Single Point of Failure - SPOF*) né collo di bottiglia centrale.
- **Protocollo Gossip**: Lo stato di appartenenza al cluster è propagato in modo asincrono tramite scambi casuali di stato tra coppie di nodi.
- **Failure Detection Distribuito**: Monitoraggio automatico e continuo della raggiungibilità dei nodi.

1.6.2 Il Protocollo Gossip e la Convergenza con Vector Clock

I nodi formano una rete di overlay. A cadenza periodica (default: 1 secondo), ciascun nodo seleziona a caso un altro membro del cluster e scambia la propria vista di appartenenza.

- **Vector Clock per la Causalità**: Ogni messaggio di gossip include un **Vector Clock** che traccia l'ordine causale parziale delle mutazioni di stato (nodi che entrano, escono o divengono non raggiungibili).
- **Convergenza del Gossip**: Un nodo raggiunge la convergenza quando può verificare matematicamente che tutti i membri raggiungibili del cluster condividono la medesima vista di appartenenza.
- **Il Ruolo del Leader**: Le transizioni di topologia (es. la promozione di un nodo da *Joining* a *Up*) possono essere decise unicamente dal **Cluster Leader** eletto, una volta che sia stata accertata la piena convergenza del gossip su tutti i nodi raggiungibili.

1.6.3 Ciclo di Vita del Nodo e Failure Detector Φ-Accrual

Uno stato di nodo nel cluster evolve formalmente secondo la sequenza:

$$\text{Joining} \longrightarrow \text{Up} \longrightarrow \text{Leaving} \longrightarrow \text{Exiting} \longrightarrow \text{Down} \longrightarrow \text{Removed} \tag{1.4}$$

Il Φ Accrual Failure Detector

Akka adotta il Φ **Accrual Failure Detector** (sviluppato da Hayashibara et al.). Anziché operare decisioni binarie (attivo/spento) basate su timeout rigidi di heartbeat, il detector raccoglie una cronologia statistica degli intervalli di inter-arrivo dei battiti cardiaci, adattandoli a una distribuzione normale. Restituisce una variabile continua di sospetto Φ:

$$\Phi = -\log_{10}(P_{\text{later}}(t - t_{\text{last}})) \tag{1.5}$$

In caso di momentanea congestione di rete, Φ sale gradualmente. Se Φ supera una soglia configurata (default: 10), il nodo viene contrassegnato come **Unreachable** (Non Raggiungibile). La presenza di nodi non raggiungibili blocca la convergenza finché il nodo non ripristina la connettività o viene marcato come **Down**.

1.6.4 Seed Node e Strumenti di Alto Livello

- **Seed Nodes**: Indirizzi di contatto preconfigurati usati come punti di rendezvous per i nuovi nodi che intendono entrare nel cluster.
- **Cluster Singleton**: Assicura che esista una sola istanza al mondo di un determinato attore all'interno dell'intero cluster, con failover trasparente in caso di crash del nodo ospitante.
- **Cluster Sharding**: Partiziona automaticamente un vasto insieme di attori con stato sui vari nodi del cluster, instradando i messaggi alla destinazione corretta sulla base dell'identificatore di entità.
- **Distributed Data**: Replica lo stato tra nodi impiegando tipi di dati privi di conflitti (*Conflict-Free Replicated Data Types - CRDT*), consentendo modifiche concorrenti prive di lock che convergono deterministicamente.

—

1.7 Analisi Critica, Complessità e Trade-off Architetturali

1.7.1 Analisi Quantitativa delle Risorse e Complessità

1.7.2 Trade-off Architetturali Fondamentali

Nella progettazione di sistemi distribuiti su Akka, gli ingegneri affrontano scelte architetturali critiche:

1. **Dimensionamento dei Dispatcher e Operazioni Bloccanti**: Gli attori condividono thread pool sottostanti (ForkJoin o ThreadPoolExecutor). Qualora un attore esegua un'operazione bloccante (es. query JDBC sincrona, chiamate I/O bloccanti), i thread del pool vengono saturati, impedendo l'esecuzione di attori del tutto indipendenti. *Soluzione*: Isolare le chiamate bloccanti su un thread pool dedicato tramite Future.
2. **Dimensionamento della Mailbox (Unbounded vs Bounded)**: Una mailbox a capacità illimitata assorbe picchi temporanei senza scartare pacchetti, ma espone al rischio di `OutOfMemoryError` in caso di saturazione cronica. Mailbox limitate (*bounded*) tutelano la memoria ma richiedono strategie esplicite di scarto (*drop*) o contropressione.

Metrica / Dimensione	**Limite Quantitativo**	**Analisi Architetturale e Trade-off**
Occupazione di Memoria	$\approx 300 - 500$ byte per istanza di Attore.	A fronte di un thread di sistema operativo (≈ 1 MB di stack), è possibile allocare milioni di attori all'interno dello heap standard della JVM.
Complessità di Trasmissione	$O(1)$ punto-a-punto; $O(N)$ multicast.	Il recapito locale equivale al passaggio diretto di un puntatore; il recapito remoto richiede una singola scrittura su socket TCP.
Latenza di Convergenza Gossip	$O(\log N)$ round in un cluster di N nodi.	La propagazione dell'informazione scala logaritmicamente con il numero di nodi; l'overhead di heartbeat è strettamente limitato.
Colli di Bottiglia Concorrenti	Zero contese di lock nell'attore.	L'unico collo di bottiglia insorge se la frequenza di arrivo dei messaggi supera stabilmente la velocità sequenziale di elaborazione dell'attore.
Costo di Ripristino da Guasto	Restart: riallocazione $O(1)$.	Il riavvio preserva la mailbox già accodata, minimizzando la perdita di messaggi durante il ripristino da guasti transitori.

Tabella 1.3: Caratteristiche Prestazionali e Consumo Risorse in Akka.

3. **Granularità della Supervisione**: Gerarchie profonde isolano precisamente il raggio di impatto dei guasti (*blast radius*), ma comportano una modesta latenza di attraversamento dell'albero gerarchico.

—

1.8 Domande di Ripasso ed Esercizi d'Esame Risolti (Politecnico di Milano)

1.8.1 Domande Teoriche di Verifica

Domanda 1: Quali sono le differenze operative tra le direttive di supervisione Resume e Restart? In quale scenario applicativo un ingegnere deve tassativamente optare per la Resume? Soluzione: All'atto dell'intercettazione di un'eccezione, se il supervisore applica la direttiva `Resume`, l'istanza dell'attore in memoria **non viene distrutta**. Il suo stato interno e tutte le sue variabili private rimangono intatti, e l'attore passa immediatamente a elaborare il messaggio successivo in coda nella mailbox.

Al contrario, con la direttiva `Restart`:

1. L'istanza dell'attore che ha sollevato l'errore viene de-allocata (invocando l'hook `preRestart(reason, message)`).
2. Viene istanziato un nuovo oggetto attore a partire dai `Props` iniziali, invocando `postRestart(reason)`.
3. Lo stato interno dell'attore viene azzerato e re-inizializzato ai valori di default del costruttore.
4. La mailbox viene preservata intatta e la nuova istanza prosegue elaborando i messaggi pendenti.

Criterio di Scelta: L'ingegnere deve impiegare tassativamente la direttiva `Resume` quando il fallimento è causato da un dato malformato o da un'anomalia esterna transitoria che non ha corrotto le strutture dati interne dell'attore (es. una lettura di temperatura non valida o un messaggio sconosciuto). In tale contesto, lo stato storico accumulato (es. la media progressiva delle misure corrette o la mappa delle sottoscrizioni dei client) è prezioso e non deve essere resettato.

Domanda 2: Per quale ragione i messaggi in Akka devono essere rigorosamente immutabili e in cosa differiscono i meccanismi interni del pattern Ask rispetto al pattern Tell? Soluzione: All'interno della medesima JVM, Akka ottimizza la comunicazione passando direttamente i riferimenti agli oggetti messaggio tra le mailbox per evitare il gravoso overhead di serializzazione. Se i messaggi fossero mutabili, più thread che servono attori distinti potrebbero accedere e modificare contemporaneamente gli stessi campi di memoria, scatenando race condition, problemi di visibilità di cache e incoerenze di stato.

L'operazione `tell()` è un'interazione asincrona *fire-and-forget*: si limita ad accodare il messaggio nella mailbox del destinatario e ritorna istantaneamente. Il pattern `ask()` crea invece un *Promise Actor* effimero di sistema, avvia un timer per il timeout prefissato, imposta tale attore fittizio come mittente e restituisce una `Future`. L'Ask comporta un maggiore consumo di risorse di memoria e scheduling e rischia di degradare le prestazioni se atteso in modo bloccante con `Await.result()`.

—

1.8.2 Soluzione Evaluation Lab 2022: Broker Pub/Sub con Partizionamento e Batching Dinamico

Specifica del Problema

Progettare un sistema Publish-Subscribe distribuito in Akka costituito da:

- **Due Worker Actor**: Verificano la corrispondenza tra eventi pubblicati e sottoscrizioni registrate.
- **Un Broker Actor**: Supervisiona e coordina i worker, partizionando le registrazioni sulla base della parità della chiave (chiavi pari assegnate al Worker 1, chiavi dispari al Worker 2).
- **Tolleranza ai Guasti**: Quando un worker riceve un messaggio `PublishMsg` per un topic mai registrato in precedenza, solleva un'eccezione. Il broker deve supervisionare i worker applicando una strategia che preservi l'insieme delle sottoscrizioni attive prima del guasto (`Resume`).
- **Batching Dinamico**: Alla ricezione di un messaggio `BatchMsg(isOn = true)`, il broker deve commutare comportamento e accumulare i messaggi di evento in arrivo. Alla ricezione di `BatchMsg(isOn = false)`, tutti gli eventi accumulati nel buffer devono essere inoltrati ed elaborati in blocco.

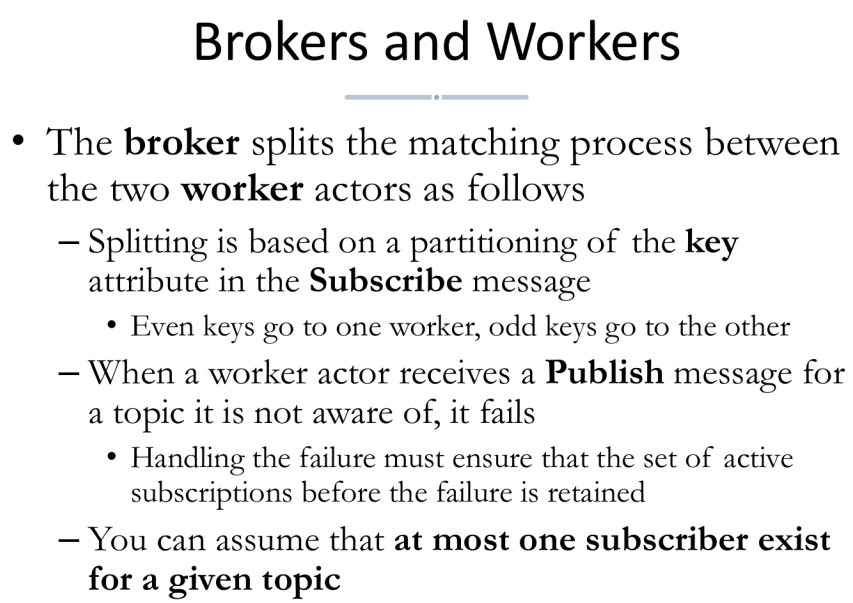


Figura 1.6: Architettura dell'Evaluation Lab 2022: Broker con partizionamento su worker pari/dispari, accumulo batch e supervisione Resume.

Implementazione Completa in Codice Java

```java
import akka.actor.ActorRef;
import java.io.Serializable;

public class Messages {
    public static class SubscribeMsg implements Serializable {
        private final String topic;
        private final int key;
        private final ActorRef subscriber;

        public SubscribeMsg(String topic, int key, ActorRef subscriber) {
            this.topic = topic;
            this.key = key;
            this.subscriber = subscriber;
        }
        public String getTopic() { return topic; }
        public int getKey() { return key; }
        public ActorRef getSubscriber() { return subscriber; }
    }

    public static class PublishMsg implements Serializable {
        private final String topic;
        private final int key;
        private final String payload;

        public PublishMsg(String topic, int key, String payload) {
            this.topic = topic;
            this.key = key;
            this.payload = payload;
        }
        public String getTopic() { return topic; }
        public int getKey() { return key; }
        public String getPayload() { return payload; }
    }

    public static class NotifyMsg implements Serializable {
        private final String payload;
        public NotifyMsg(String payload) { this.payload = payload; }
        public String getPayload() { return payload; }
    }

    public static class BatchMsg implements Serializable {
        private final boolean on;
        public BatchMsg(boolean on) { this.on = on; }
        public boolean isOn() { return on; }
    }
}
```

Listing 1.1: Definizione dei Messaggi del Sistema Pub/Sub

```java
import akka.actor.AbstractActor;
import akka.actor.Props;
import java.util.HashMap;
import java.util.Map;

public class WorkerActor extends AbstractActor {
    // Mappa delle sottoscrizioni: Topic -> Riferimento Sottoscrittore
    private final Map<String, ActorRef> subscriptions = new HashMap<>();

    public static Props props() {
        return Props.create(WorkerActor.class, WorkerActor::new);
```

```
    }

    @Override
    public Receive createReceive() {
        return receiveBuilder()
            .match(Messages.SubscribeMsg.class, msg -> {
                subscriptions.put(msg.getTopic(), msg.getSubscriber());
                System.out.println(self().path().name() + " ha registrato
    il topic: " + msg.getTopic());
            })
            .match(Messages.PublishMsg.class, msg -> {
                if (!subscriptions.containsKey(msg.getTopic())) {
                    System.err.println(self().path().name() + " fallimento:
     topic sconosciuto " + msg.getTopic());
                    throw new Exception("Topic sconosciuto: " + msg.
    getTopic());
                }
                ActorRef sub = subscriptions.get(msg.getTopic());
                sub.tell(new Messages.NotifyMsg(msg.getPayload()), self());
            })
            .build();
    }
}
```

Listing 1.2: WorkerActor con Gestione Guasti su Topic Non Registrati

```
import akka.actor.*;
import akka.japi.pf.DeciderBuilder;
import java.time.Duration;
import java.util.LinkedList;
import java.util.Queue;

public class BrokerActor extends AbstractActorWithStash {
    private final ActorRef workerEven;
    private final ActorRef workerOdd;
    private final Queue<Messages.PublishMsg> batchBuffer = new LinkedList
    <>();

    // Politica di supervisione: Resume per preservare le registrazioni
    attive
    private static final SupervisorStrategy STRATEGY = new
    OneForOneStrategy(
        10,
        Duration.ofMinutes(1),
        DeciderBuilder
            .match(Exception.class, e -> SupervisorStrategy.resume())
            .build()
    );

    @Override
    public SupervisorStrategy supervisorStrategy() {
        return STRATEGY;
    }

    public BrokerActor() {
        this.workerEven = getContext().actorOf(WorkerActor.props(), "
    workerEven");
        this.workerOdd = getContext().actorOf(WorkerActor.props(), "
    workerOdd");
    }

```

```
31      public static Props props() {
32          return Props.create(BrokerActor.class, BrokerActor::new);
33      }
34
35      @Override
36      public Receive createReceive() {
37          return normalBehavior();
38      }
39
40      // Comportamento Ordinario: instradamento immediato
41      private Receive normalBehavior() {
42          return receiveBuilder()
43              .match(Messages.BatchMsg.class, msg -> {
44                  if (msg.isOn()) {
45                      System.out.println("Broker: passaggio alla modalita
    BATCH.");
46                      getContext().become(batchBehavior());
47                  }
48              })
49              .match(Messages.SubscribeMsg.class, this::routeSubscription)
50              .match(Messages.PublishMsg.class, this::routePublish)
51              .build();
52      }
53
54      // Comportamento Batch: accumulo degli eventi nel buffer
55      private Receive batchBehavior() {
56          return receiveBuilder()
57              .match(Messages.BatchMsg.class, msg -> {
58                  if (!msg.isOn()) {
59                      System.out.println("Broker: rilascio batch (" +
    batchBuffer.size() + " messaggi).");
60                      while (!batchBuffer.isEmpty()) {
61                          routePublish(batchBuffer.poll());
62                      }
63                      getContext().become(normalBehavior());
64                  }
65              })
66              .match(Messages.SubscribeMsg.class, this::routeSubscription)
67              .match(Messages.PublishMsg.class, batchBuffer::add)
68              .build();
69      }
70
71      private void routeSubscription(Messages.SubscribeMsg msg) {
72          if (msg.getKey() % 2 == 0) {
73              workerEven.forward(msg, getContext());
74          } else {
75              workerOdd.forward(msg, getContext());
76          }
77      }
78
79      private void routePublish(Messages.PublishMsg msg) {
80          if (msg.getKey() % 2 == 0) {
81              workerEven.forward(msg, getContext());
82          } else {
83              workerOdd.forward(msg, getContext());
84          }
85      }
86  }
```

Listing 1.3: BrokerActor con Switch Dinamico del Comportamento e Supervisione Resume

—

1.8.3 Soluzione Evaluation Lab 2023: Monitoraggio Sensori con Criteri di Smistamento

Specifica del Problema

Sviluppare un'architettura in Akka per il monitoraggio di sensori industriali composta da:

1. **Attori SensorDataProcessor**: Mantengono la media progressiva delle letture di temperatura ricevute:
$$\bar{T}_k = \frac{(k-1)\bar{T}_{k-1} + T_k}{k} \tag{1.6}$$
Se un processore riceve un valore di temperatura negativo ($T < 0$), solleva un'eccezione. La gestione del guasto deve garantire che la media calcolata fino a quel momento venga preservata e che la lettura errata venga scartata dal computo (`Resume`).

2. **Un Attore DispatcherActor**: Instrada dinamicamente le letture in arrivo secondo due strategie selezionabili via messaggio `DispatchLogicMsg`:
   - **Load Balancing**: Associa stabilmente ciascun sensore al processore che gestisce il minor numero di sensori, garantendo bilanciamento del carico.
   - **Round Robin**: Distribuisce a rotazione circolare ogni nuova lettura ricevuta tra i processori disponibili.

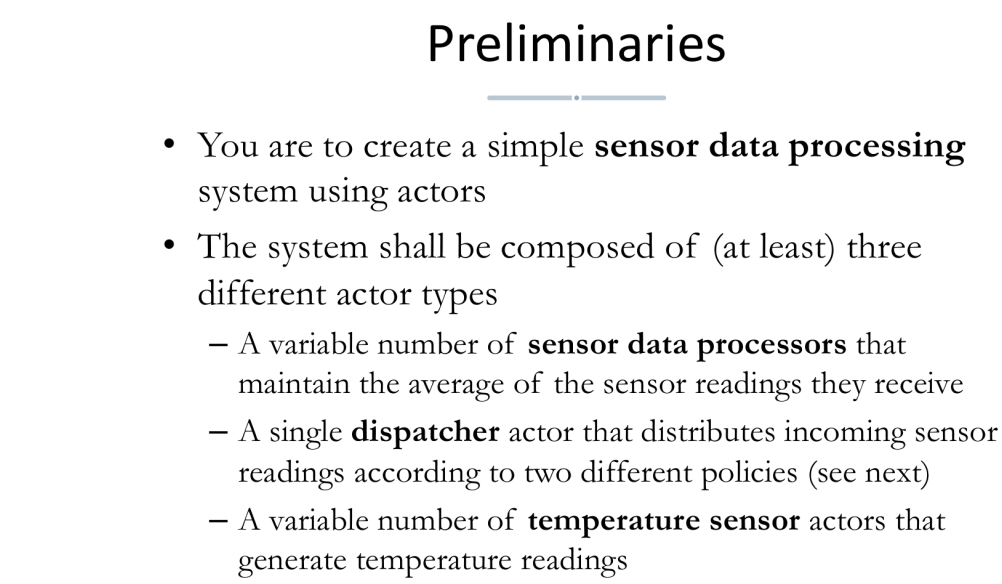


Figura 1.7: Architettura dell'Evaluation Lab 2023: Dispatcher con switch di smistamento Round Robin / Load Balancing e processori resilienti con Resume.

Implementazione Completa in Codice Java

```
1 import akka.actor.AbstractActor;
2 import akka.actor.Props;
3
```

```
public class SensorProcessorActor extends AbstractActor {
    private double sum = 0.0;
    private int count = 0;

    public static Props props() {
        return Props.create(SensorProcessorActor.class,
    SensorProcessorActor::new);
    }

    @Override
    public Receive createReceive() {
        return receiveBuilder()
            .match(TemperatureMsg.class, msg -> {
                if (msg.getValue() < 0) {
                    System.err.println(self().path().name() + " rifiuta
    temperatura non valida: " + msg.getValue());
                    throw new IllegalArgumentException("Temperatura
    negativa non ammessa: " + msg.getValue());
                }
                count++;
                sum += msg.getValue();
                double currentAvg = sum / count;
                System.out.printf("%s -> Misura: %.2f | Nuova Media: %.2f (
    Campioni totali: %d)%n",
                    self().path().name(), msg.getValue(), currentAvg, count
    );
            })
            .build();
    }
}
```

Listing 1.4: SensorProcessorActor con Preservazione della Media Storica

```
import akka.actor.*;
import akka.japi.pf.DeciderBuilder;
import java.time.Duration;
import java.util.*;

public class DispatcherActor extends AbstractActor {
    private final List<ActorRef> processors;
    private final Map<ActorRef, ActorRef> sensorToProcessorMap = new
    HashMap<>();
    private final Map<ActorRef, Integer> processorLoadMap = new HashMap<>()
    ;
    private int roundRobinIndex = 0;
    private boolean isRoundRobin = true; // Politica di default

    private static final SupervisorStrategy STRATEGY = new
    OneForOneStrategy(
        10,
        Duration.ofMinutes(1),
        DeciderBuilder
            .match(IllegalArgumentException.class, e -> SupervisorStrategy.
    resume())
            .build()
    );

    @Override
    public SupervisorStrategy supervisorStrategy() {
        return STRATEGY;
    }
```

```
25
26      public DispatcherActor(int numProcessors) {
27          this.processors = new ArrayList<>();
28          for (int i = 0; i < numProcessors; i++) {
29              ActorRef p = getContext().actorOf(SensorProcessorActor.props(),
     "processor-" + i);
30              processors.add(p);
31              processorLoadMap.put(p, 0);
32          }
33      }
34
35      public static Props props(int numProcessors) {
36          return Props.create(DispatcherActor.class, () -> new
    DispatcherActor(numProcessors));
37      }
38
39      @Override
40      public Receive createReceive() {
41          return receiveBuilder()
42              .match(DispatchLogicMsg.class, msg -> {
43                  this.isRoundRobin = msg.isRoundRobin();
44                  System.out.println("Dispatcher: logica commutata in " +
45                      (isRoundRobin ? "ROUND ROBIN" : "LOAD BALANCING"));
46              })
47              .match(TemperatureMsg.class, msg -> {
48                  ActorRef targetProcessor;
49                  if (isRoundRobin) {
50                      targetProcessor = processors.get(roundRobinIndex);
51                      roundRobinIndex = (roundRobinIndex + 1) % processors.
    size();
52                  } else {
53                      ActorRef sensorSender = sender();
54                      if (!sensorToProcessorMap.containsKey(sensorSender)) {
55                          // Individua il processore con il carico minore di
    sensori
56                          targetProcessor = processors.stream()
57                              .min(Comparator.comparingInt(processorLoadMap::
    get))
58                              .orElse(processors.get(0));
59                          sensorToProcessorMap.put(sensorSender,
    targetProcessor);
60                          processorLoadMap.put(targetProcessor,
    processorLoadMap.get(targetProcessor) + 1);
61                      } else {
62                          targetProcessor = sensorToProcessorMap.get(
    sensorSender);
63                      }
64                  }
65                  targetProcessor.forward(msg, getContext());
66              })
67              .build();
68      }
69  }
```

Listing 1.5: DispatcherActor con Politiche di Routing e Strategia di Supervisione